Attention & Transformers

The Architecture That Powers Modern AI

1

Part 1: The Problem with Sequential Data

2

Sequential Data is Everywhere

- **Text**: "The cat sat on the mat" — word order matters!
- **Speech/Audio**: Sound waveforms over time
- **Video**: Sequence of image frames
- **Time Series**: Stock prices, sensor readings
- **Key Challenge**: How do we process data where **context** and **order** matter?

3

The Old Approach: Process One at a Time

- **RNNs/LSTMs** (pre-2017): Process tokens one by one, left to right
- Pass a "hidden state" from each step to the next
- **Problem 1**: Very slow — can't parallelize (must wait for previous step)
- **Problem 2**: Long-range dependencies get "forgotten" over many steps
- **Problem 3**: Vanishing gradients make training difficult
- We needed something better...

4

The Breakthrough: Attention Is All You Need

- **2017**: Vaswani et al. introduce the **Transformer** architecture
- **Key insight**: Instead of processing sequentially, let every position "attend" to every other position **in parallel**
- No recurrence, no convolutions — just attention
- **Result**: Faster training, better at capturing long-range dependencies
- This paper changed everything: GPT, BERT, LLaMA, Gemini — all Transformers!

5

Part 2: The Attention Mechanism

6

What is Attention?

- **Intuition**: When reading, you focus on relevant words to understand context
- "The **cat** sat on the mat. **It** was tired." — What does "it" refer to?
- **Attention** lets the model learn which parts of the input are relevant to each output
- It computes a **weighted combination** of all inputs, where weights depend on relevance

7

The Query-Key-Value Framework

- Think of it like a **database lookup**:
- **Query (Q)**: "What am I looking for?" — the current position asking a question
- **Key (K)**: "What do I contain?" — each position advertising its content
- **Value (V)**: "What information do I provide?" — the actual content to retrieve
- **Process**: Compare Query with all Keys → Get relevance scores → Weighted sum of Values

8

Scaled Dot-Product Attention

- $$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$
- **Step 1**: Compute similarity scores: QK^T (dot product of queries and keys)
- **Step 2**: Scale by $\sqrt{d_k}$ (prevents extremely large values)
- **Step 3**: Softmax to get attention weights (probabilities that sum to 1)
- **Step 4**: Weighted sum of Values

🚀 **Interactive Demo:** attention_demo.html

9

Self-Attention: Attending to Yourself

- **Self-attention**: Q, K, V all come from the **same** input sequence
- Each position can attend to all other positions (including itself)
- **Example**: In "The cat sat on the mat", the word "sat" can directly look at "cat", "mat", etc.
- This captures relationships regardless of distance in the sequence!
- Unlike RNNs: No information bottleneck, no sequential processing

10

Self-Attention in PyTorch

PYTHON

```python
import torch
import torch.nn.functional as F

def self_attention(x, W_q, W_k, W_v):
    """x: (batch, seq_len, d_model)"""
    Q = x @ W_q  # Queries
    K = x @ W_k  # Keys
    V = x @ W_v  # Values

    d_k = K.shape[-1]
    scores = (Q @ K.transpose(-2, -1)) / (d_k ** 0.5)
    attn_weights = F.softmax(scores, dim=-1)
    output = attn_weights @ V

    return output, attn_weights
```

Part 3: Multi-Head Attention

12

Why Multiple Heads?

- **Problem**: A single attention can only focus on one type of relationship
- **Solution**: Use **multiple attention heads** in parallel
- Each head learns to focus on **different aspects**:
- Head 1 might focus on syntax (subject-verb)
- Head 2 might focus on coreference ("it" → "cat")
- Head 3 might focus on semantic similarity
- **Combine**: Concatenate all heads, then project back

13

Multi-Head Attention Formula

- $$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W^O$$
- $$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$
- Each head has its own Q, K, V projection matrices
- **Typical**: 8-16 heads in base models, 32+ in large models
- **Key insight**: More heads = more diverse attention patterns

14

Multi-Head Attention in PyTorch

PYTHON

```
class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, num_heads):
        super().__init__()
        self.num_heads = num_heads
        self.d_k = d_model // num_heads

        self.W_q = nn.Linear(d_model, d_model)
        self.W_k = nn.Linear(d_model, d_model)
        self.W_v = nn.Linear(d_model, d_model)
        self.W_o = nn.Linear(d_model, d_model)

    def forward(self, x):
        B, L, D = x.shape
        # Project and reshape to (B, num_heads, L, d_k)
        Q = self.W_q(x).view(B, L, self.num_heads, self.d_k).transp
        K = self.W_k(x).view(B, L, self.num_heads, self.d_k).transp
        V = self.W_v(x).view(B, L, self.num_heads, self.d_k).transp

        # Scaled dot-product attention
        scores = (Q @ K.transpose(-2, -1)) / (self.d_k ** 0.5)
        attn = F.softmax(scores, dim=-1)
        out = (attn @ V).transpose(1, 2).reshape(B, L, D)

        return self.W_o(out)
```

Part 3.5: PyTorch Built-in Layers

16

Using nn.MultiheadAttention

- PyTorch provides **built-in attention layers** — use them in production!
- **nn.MultiheadAttention**: Optimized multi-head attention implementation
- Handles all the complexity: Q/K/V projections, head splitting, masking
- **Key parameters**: embed_dim, num_heads, dropout, batch_first
- Returns both output and attention weights

17

Using nn.MultiheadAttention

PYTHON

```
import torch.nn as nn

# Create the layer
mha = nn.MultiheadAttention(
    embed_dim=512,     # Model dimension
    num_heads=8,       # Number of attention heads
    dropout=0.1,       # Dropout rate
    batch_first=True   # Input shape: (batch, seq, dim)
)

# Forward pass (self-attention: Q=K=V)
x = torch.randn(2, 10, 512)  # batch=2, seq=10
output, attn_weights = mha(x, x, x)
# output: (2, 10, 512)
# attn_weights: (2, 10, 10)
```

18

From Scratch vs Built-in: When to Use Each

- **From Scratch** (for learning):
- • Understand exactly what's happening inside
- • Customize behavior (research, experiments)
- • Required for assignments and interviews!
- **Built-in Layers** (for production):
- • Optimized for speed (fused CUDA kernels)
- • Well-tested and maintained
- • Handles edge cases (masking, padding)
- **Best practice**: Understand from scratch, deploy with built-in

19

Complete Comparison

PYTHON

```python
# FROM SCRATCH (what you implement in assignments)
class SelfAttention(nn.Module):
    def __init__(self, d_model):
        self.W_q = nn.Linear(d_model, d_model)
        self.W_k = nn.Linear(d_model, d_model)
        self.W_v = nn.Linear(d_model, d_model)

    def forward(self, x):
        Q, K, V = self.W_q(x), self.W_k(x), self.W_v(x)
        scores = Q @ K.transpose(-2, -1) / (Q.size(-1) ** 0.5)
        return F.softmax(scores, dim=-1) @ V

# BUILT-IN (what you use in production)
self.attn = nn.MultiheadAttention(d_model, num_heads)
output, _ = self.attn(x, x, x)  # Same result, optimized!
```

20

Part 4: Positional Encoding

21

The Position Problem

- **Issue**: Self-attention treats input as a **set**, not a sequence!
- "Cat ate mouse" vs "Mouse ate cat" → same attention outputs without position info
- **Solution**: Add **positional encoding** to the input embeddings
- Inject information about where each token is in the sequence

22

Sinusoidal Positional Encoding

- Original Transformer uses sine and cosine functions:
- $$PE_{(pos,2i)} = \sin(pos/10000^{2i/d})$$
- $$PE_{(pos,2i+1)} = \cos(pos/10000^{2i/d})$$
- **Why?** Different frequencies allow the model to attend to relative positions
- **Modern approach**: Learned positional embeddings (just train them!)
- **RoPE** (Rotary Position Embedding): Used in LLaMA, GPT-NeoX

23

Part 5: The Transformer Block

24

Inside a Transformer Block

- Each block has two sub-layers:
- **1. Multi-Head Self-Attention**: Lets tokens communicate
- **2. Feed-Forward Network (FFN)**: Processes each token independently
- Both have **residual connections** and **layer normalization**
- Stack many blocks (6-96+) for deeper understanding

🚀 **Interactive Demo:** transformer_block_demo.html

25

Residual Connections & Layer Norm

- **Residual connections**: $\text{output} = x + \text{SubLayer}(x)$
- Helps gradients flow through deep networks (like in ResNet)
- **Layer Normalization**: Normalize across features for each token
- $$\text{LayerNorm}(x) = \gamma \cdot \frac{x - \mu}{\sigma} + \beta$$
- **Pre-norm vs Post-norm**: Modern models often use Pre-norm (more stable)

26

Feed-Forward Network

- Applied to each position **independently** (no interaction between tokens)
- Simple two-layer MLP with expansion:
- $$\mathrm{FFN}(x) = \mathrm{GELU}(xW_1 + b_1)W_2 + b_2$$
- **Expansion factor**: Usually 4× (e.g., 768 → 3072 → 768)
- This is where individual token representations get refined

27

Transformer Block in PyTorch

PYTHON

```python
class TransformerBlock(nn.Module):
    def __init__(self, d_model, num_heads, d_ff):
        super().__init__()
        self.attn = MultiHeadAttention(d_model, num_heads)
        self.ffn = nn.Sequential(
            nn.Linear(d_model, d_ff),
            nn.GELU(),
            nn.Linear(d_ff, d_model)
        )
        self.ln1 = nn.LayerNorm(d_model)
        self.ln2 = nn.LayerNorm(d_model)

    def forward(self, x):
        # Pre-norm style
        x = x + self.attn(self.ln1(x))
        x = x + self.ffn(self.ln2(x))
        return x
```

Part 6: Encoder vs Decoder

29

Two Flavors of Transformers

- **Encoder-only** (e.g., BERT):
- • Bidirectional: Each token sees all other tokens
- • Great for understanding (classification, NER, embeddings)
- **Decoder-only** (e.g., GPT, LLaMA):
- • Causal/autoregressive: Token can only see previous tokens
- • Great for generation (text, code, reasoning)
- **Encoder-Decoder** (e.g., T5, original Transformer): For seq2seq tasks

30

Causal Masking (for Decoders)

- **Problem**: During training, we give the model the full sequence
- But at inference, it only has tokens generated so far!
- **Solution**: Mask future tokens during training
- Before softmax, set future positions to $-\infty$ → softmax gives 0 weight
- $$\text{mask}[i][j] = \begin{cases} 0 & \text{if } j \leq i \\ -\infty & \text{if } j > i \end{cases}$$
- This is how GPT trains on next-token prediction

31

Causal Mask Implementation

PYTHON

```
def causal_attention(x, W_q, W_k, W_v):
    Q, K, V = x @ W_q, x @ W_k, x @ W_v
    d_k = K.shape[-1]

    scores = (Q @ K.transpose(-2, -1)) / (d_k ** 0.5)

    # Create causal mask
    seq_len = x.shape[1]
    mask = torch.triu(torch.ones(seq_len, seq_len), diagonal=1).boo
    scores = scores.masked_fill(mask, float('-inf'))

    attn = F.softmax(scores, dim=-1)
    return attn @ V
```

32

Part 7: Putting It All Together

33

Full Transformer Architecture

- **Input**: Token IDs → Token Embeddings + Positional Encoding
- **Stack**: N Transformer Blocks (each with Attention + FFN)
- **Output**: Contextual representations for each token
- **For generation**: Add output projection to vocabulary size
- **Scale**: GPT-3 = 96 blocks, 12288 dims, 96 heads, 175B parameters

34

Why Transformers Dominate

- **Parallelization**: All positions computed simultaneously (fast on GPUs)
- **Long-range dependencies**: Direct path between any two positions
- **Scalability**: Performance improves predictably with more data/compute
- **Flexibility**: Same architecture for text, images, audio, code, proteins
- **Transfer learning**: Pre-train once, fine-tune for anything

35

PyTorch's nn.Transformer

- PyTorch provides **complete Transformer** implementations!
- **nn.TransformerEncoder**: Stack of encoder layers
- **nn.TransformerDecoder**: Stack of decoder layers
- **nn.Transformer**: Full encoder-decoder architecture
- Use these for production, understand the internals for research

36

Building a Transformer in PyTorch

PYTHON

```
import torch.nn as nn

class SimpleTransformer(nn.Module):
    def __init__(self, vocab_size, d_model=512, nhead=8, num_layers
        super().__init__()
        self.embed = nn.Embedding(vocab_size, d_model)
        self.pos_enc = PositionalEncoding(d_model)  # You implement

        # Use built-in TransformerEncoder
        encoder_layer = nn.TransformerEncoderLayer(
            d_model=d_model, nhead=nhead, batch_first=True
        )
        self.transformer = nn.TransformerEncoder(encoder_layer, num
        self.fc_out = nn.Linear(d_model, vocab_size)

    def forward(self, x):
        x = self.embed(x)
        x = self.pos_enc(x)
        x = self.transformer(x)
        return self.fc_out(x)
```

37

All Interactive Demos

- 📊 **Attention Visualizations:**

  🚀 **Interactive Demo:** attention_demo.html

- 🔲 **Transformer Block:**

  🚀 **Interactive Demo:** transformer_block_demo.html

- 🔧 **From Scratch vs Built-in:**

  🚀 **Interactive Demo:** pytorch_attention_demo.html

- 🏗️ **Full Transformer Architecture:**

  🚀 **Interactive Demo:** full_transformer_demo.html

38

Summary

39

Key Takeaways

- **Attention**: Weighted combination based on relevance (Query-Key-Value)
- **Self-Attention**: Every position attends to every other position
- **Multi-Head**: Multiple attention patterns in parallel
- **Positional Encoding**: Inject sequence order information
- **Transformer Block**: Attention + FFN with residuals and layer norm
- **PyTorch Built-ins**: nn.MultiheadAttention, nn.TransformerEncoder
- **Practice**: Build from scratch to learn, use built-in for production

40